Benchmarking SLH-DSA: A Comparative Hardware Analysis Against Classical Digital Signatures for Post-Quantum Security

Jayalaxmi H, H M Brunda,, Sumith Subraya Nayak, Sathya M, Anirudh S Hegde
Department. Of ECE , Acharya Institute of Technology,
Bengaluru, India
hh.22.beec@acharya.ac.in,jayalaxmi@acharya.ac.in ,
sumithnayak07@gmail.com

***Abstract*—The imminent threat posed by large-scale quantum computers necessitates a paradigm shift from classical public-key cryptography to quantum-resistant solutions. In response, the National Institute of Standards and Technology (NIST) has standardized several Post-Quantum Cryptography (PQC) algorithms, including the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA), specified in FIPS 205. The practical adoption of SLH-DSA, particularly in hardware-constrained environments such as embedded systems and Root-of-Trust (RoT) modules, depends critically on a comprehensive understanding of its performance and resource overhead relative to legacy standards. This paper presents a definitive hardware benchmarking study, implementing and synthesizing Verilog HDL designs for SLH-DSA and a suite of classical signature schemes—RSA, DSA, ECDSA, and EdDSA—on a unified Xilinx FPGA platform. Our comparative analysis focuses on key hardware metrics: resource utilization (LUTs, FFs, BRAMs, DSPs) and performance characteristics (latency for key generation, signing, and verification; and overall throughput). The results quantify the significant trade-offs inherent in SLH-DSA; it exhibits substantially higher signing latency and produces larger signatures than its classical counterparts. However, its verification performance is highly competitive, and its security is conservatively based on well-understood hash functions. The hardware architecture of SLH-DSA is logic- and memory-intensive, contrasting sharply with the DSP-heavy nature of RSA and ECC. We conclude that while SLH-DSA presents a computationally expensive option, its robust security model makes it a viable solution for applications prioritizing long-term security assurance over raw signing performance, such as firmware signing and digital archiving.**

***Index Terms*—Post-Quantum Cryptography (PQC), SLH-DSA, SPHINCS+, FPGA, Xilinx, Hardware Acceleration, Digital Signa- ture, RSA, DSA, ECDSA, EdDSA, Performance Benchmarking, Resource Utilization, Latency, Throughput.**

I. Introduction

The foundations of modern digital security, built upon public-key cryptosystems like RSA and the Elliptic Curve Digital Signature Algorithm (ECDSA), are threatened by the advent of fault-tolerant quantum computers. Shor's algorithm, when executed on such a machine, can solve the integer factorization and discrete logarithm problems in polynomial time, rendering these widely deployed standards obsolete. This impending reality has catalyzed a global, proactive migration toward Post-Quantum Cryptography (PQC), a new class of algorithms designed to be secure against attacks from both classical and quantum computers.

Spearheading this transition is the U.S. National Institute of Standards and Technology (NIST), which has conducted a multi-year, multi-round public process to solicit, evaluate, and standardize PQC algorithms. This effort recently culmi-nated in the publication of the first PQC standards, including the Stateless Hash-Based Digital Signature Algorithm (SLH- DSA), formerly known as SPHINCS+, which is formalized in FIPS 205. SLH-DSA offers a unique security proposition among the new standards. Its security does not rely on the presumed difficulty of relatively new mathematical problems, such as those in lattice-based cryptography, but is instead conservatively grounded in the collision and preimage resistance of its underlying cryptographic hash functions, like SHA-2 or SHAKE. This reliance on well-vetted cryptographic primitives provides a high degree of confidence in its long-term security. However, the theoretical security of an algorithm is only one facet of its viability. For PQC standards to be successfully deployed in real-world systems—especially in resource-constrained environments like Internet of Things (IoT) devices, automotive electronics, and hardware security modules (HSMs)—a rigorous evaluation of their hardware implementation characteristics is paramount. Software benchmarks and theoretical complexity analyses are insufficient as they fail to capture the nuances of hardware resource trade-offs, archi- tectural suitability, and achievable performance on physical silicon.

This work provides a direct comparative hardware analysis of a fully-featured SLH-DSA implementation against a comprehensive suite of classical digital signature standards (RSA, DSA, ECDSA, and EdDSA) on a unified Xilinx FPGA platform. By synthesizing and benchmarking all algorithms under identical conditions, this study offers a clear, quantitative assessment of the costs and benefits associated with migrating to hash-based PQC signatures. The subsequent sections will review the existing literature on hardware implementations, detail our benchmarking methodology, present and discuss the results of our comparative analysis, and conclude with the implications for future secure system design.

II. Literature Review: The State of Digital Signature Implementation

The transition to a post-quantum cryptographic landscape necessitates a thorough understanding of not only the new PQC standards but also the well-established hardware perfor- mance profiles of the classical algorithms they are poised to replace. This review surveys the state of the art in hardware implementations for both classical and post-quantum digital signatures, identifying the research gap this paper aims to fill.

A. Hardware Implementations of Classical Signatures

1) RSA on FPGAs: Hardware implementations of the RSA algorithm have been extensively studied. The core computational task is modular exponentiation, an operation on very large integers (e.g., 2048 bits) that is computationally intensive. To manage this complexity, hardware accelerators almost universally employ the Montgomery multiplication algorithm, which replaces costly trial division with a series of more efficient shifts and additions. For signature generation (decryption), performance is often further enhanced using the Chinese Remainder Theorem (CRT), which breaks the single large exponentiation into two smaller ones. Architec- turally, RSA implementations on FPGAs are characterized by their heavy reliance on Digital Signal Processor (DSP) slices to build large, parallel multipliers, supplemented by Block RAM (BRAM) for storing intermediate values. Performance is typically measured in terms of throughput, with reported figures varying based on key size and the level of architectural optimization.

2) Elliptic Curve Cryptography (ECDSA & EdDSA) on FPGAs: Elliptic Curve Cryptography (ECC) emerged as a more efficient alternative to RSA, offering equivalent security with significantly smaller key sizes and, consequently, faster operations. The hardware implementation literature for ECC is vast, covering a spectrum of designs from high-performance, parallel architectures to lightweight, area-efficient cores for constrained devices. The fundamental operation in ECC is scalar multiplication, which involves repeated point addition and point doubling on the curve. To protect against side-channel attacks, many secure implementations utilize the Montgomery Ladder algorithm, which performs a constant sequence of operations regardless of the secret key bits.

A key evolution in ECC signatures is the development of the Edwards-curve Digital Signature Algorithm (EdDSA), exemplified by Ed25519. EdDSA improves upon ECDSA by using twisted Edwards curves, which feature complete addition formulas that eliminate exceptional points and simplify the logic. Furthermore, EdDSA generates the per-signature nonce deterministically from the message and private key, removing a major source of implementation error that has plagued ECDSA. Hardware benchmarks demonstrate that EdDSA architectures can achieve very high throughput, often measured in thousands of signatures per second on modern FPGAs, while maintaining a compact resource footprint. Like RSA, ECC implementations are characterized by their use of DSPs for the underlying finite field arithmetic.

B. Hardware Acceleration of Post-Quantum Signatures

1) Architectural Insights into Hash-Based Signatures: Hash-based signatures represent one of the oldest and most trusted families of post-quantum schemes. Their security relies solely on the properties of the underlying hash function. Early schemes were stateful, such as the Leighton-Micali Signature (LMS) and the eXtended Merkle Signature Scheme (XMSS), which require the signer to meticulously track used one-time keys to prevent catastrophic security failures. This fragility made them unsuitable for many applications.

The breakthrough came with stateless schemes like SPHINCS and its successor, SPHINCS+, which was standardized as SLH-DSA. These algorithms eliminate the need for state management at the cost of larger signatures and slower performance. Architecturally, the core of any hash-based signature scheme is the repetitive computation of a cryptographic hash function. This inherent parallelism makes them exceptionally well-suited for pipelined hardware architectures, where multiple hash instances can be computed concurrently.

2) Optimizing SLH-DSA (SPHINCS+): Initial hardware implementations of SPHINCS+ demonstrated its feasibility but also confirmed its significant performance cost compared to classical algorithms. The key to making SLH-DSA practical in hardware lies in specialized acceleration. A pivotal study demonstrated that while using a generic, memory-mapped hash accelerator can provide a roughly tenfold speedup over a software-only implementation, this approach is suboptimal [2]. SLH-DSA spends most of its time not hashing the external message, but performing internal hashing operations for generating Winternitz One-Time Signature Plus (WOTS+) chains and pseudorandom values via a PRF.

A specialized accelerator, such as the ”SLotH” prototype, is designed to offload these internal, repetitive data formatting and hashing tasks entirely to hardware [2]. For instance, computing a WOTS+ chain involves iteratively hashing a value, incrementing a counter, and re-hashing. In a software-driven approach, this requires hundreds of CPU cycles per iteration to move data in and out of the hash unit. In a specialized core, this padding and iteration logic is implemented with simple multiplexers and counters, executing in a single clock cycle. This architectural optimization leads to performance gains of up to 300x over unaccelerated microcontroller implementations [2]. Subsequent research, such as the SPHINCSLET project, has focused on creating area-efficient accelerators, demonstrating that robust PQC security can be achieved even in highly resource-constrained FPGAs [9].

C. Existing Comparative Analyses and Gaps

Several important studies have benchmarked the NIST PQC candidates against each other [4], [12]. These papers compare lattice-based schemes like CRYSTALS-Dilithium and Falcon against the hash-based SPHINCS+, analyzing the trade-offs in key/signature size, software and hardware performance, and underlying security assumptions. These works establish that lattice-based schemes are generally much faster and have

smaller signatures than SPHINCS+, but SPHINCS+ is valued for its conservative security and algorithmic diversity.

However, a clear gap remains in the literature. While many papers benchmark classical algorithms and others compare PQC candidates, a direct, unified hardware comparison of the newly finalized SLH-DSA standard against a comprehensive suite of the classical algorithms it is designed to replace (RSA, DSA, ECDSA, and EdDSA) is not readily available. Such a study is essential for system architects and engineers who must make informed decisions about migrating legacy systems. This paper provides that definitive benchmark, evaluating all algorithms on the same Xilinx FPGA platform using a consistent RTL design methodology and a uniform set of metrics.

III. Benchmarking Methodology

To provide a rigorous and equitable comparison, this study employs a unified hardware implementation and evaluation framework. All digital signature algorithms were implemented from their specifications using a consistent design methodology and targeted to the same FPGA platform. The metrics for comparison were chosen to reflect the primary concerns of hardware engineers: resource cost and operational performance.

A. Algorithmic Foundations

1) The SLH-DSA (FIPS 205) Protocol: SLH-DSA is a stateless hash-based signature scheme built from several

2) hierarchical components, as specified in NIST FIPS 205 [1]. Its security is derived entirely from the properties of an underlying cryptographic hash function, such as SHAKE256 or SHA-256. The algorithm's structure can be deconstructed as follows:

- **WOTS+ (Winternitz One-Time Signature Plus):** At the lowest level, WOTS+ serves as the one-time signature scheme. A private key consists of a set of secret random values. Each secret is hashed repeatedly (e.g., $w-1$ times, where w is the Winternitz parameter) to form a hash chain. The public key is derived from the final values in these chains. To sign a message, parts of the message are used as indices to select and reveal specific intermediate nodes from these hash chains [10], [13].
- **FORS (Forest of Random Subsets):** To sign the message digest, SLH-DSA uses FORS, a few-time signature scheme. A FORS key pair consists of a forest of k Merkle trees. Bits from the message digest are used to select a specific leaf from each of the k trees. The signature consists of these k leaf nodes and their corresponding authentication paths to the tree roots. The FORS public key is a hash of all the Merkle tree roots [10], [13].
- **Hypertree:** To sign more than a few messages, SLH- DSA organizes a vast number of FORS public keys into a massive structure called a hypertree. This is a tree of trees, constructed from multiple layers of the eXtended Merkle Signature Scheme (XMSS). The leaves of the lowest layer of XMSS trees are WOTS+ public keys, which are used to sign the FORS public keys. The root of each XMSS tree in a given layer is, in turn, signed by a WOTS+ key from the layer above it. This continues until a single XMSS tree at the top layer is reached. The root of this top-level tree serves as the single, compact public key for the entire SLH-DSA instance.

The **key generation** process involves generating the master secret seeds (SK.seed, SK.prf) and the public seed (PK.seed), and then computing the root of the top-level XMSS tree to form PK.root. **Signature generation** is a complex process: a message digest is computed, a specific FORS key pair is selected based on the digest and a counter, the digest is signed using FORS, and the resulting FORS public key is then signed using the hypertree. This involves generating one WOTS+ signature and its authentication path at each level of the tree. **Signature verification** reverses this process: the verifier recomputes the message digest, uses the FORS signature to reconstruct the FORS public key, and then uses the hypertree signature to verify the chain of signatures up to the known public root of the top-level tree. For this study, we implement
the SLH-DSA-SHAKE-128f parameter set, which is optimized for fast signing performance at a ~128-bit quantum security level.

3) Classical Algorithms for Comparison: To provide a comprehensive benchmark, we implemented the following widely used classical digital signature algorithms:

- **RSA:** Based on the difficulty of factoring large integers. The core operations are modular exponentiation, implemented using Montgomery multiplication for efficiency. We use a 2048-bit modulus.
- **DSA:** The original Digital Signature Algorithm, based on the discrete logarithm problem in a finite field ($\mathbb{Z}^*$). Its operations are also modular exponentiations. We use a
2048-bit modulus.
- **ECDSA:** Based on the elliptic curve discrete logarithm problem (ECDLP). Its core operation is scalar multiplication on an elliptic curve. We use the 256-bit NIST curve P-256 (secp256r1).
- **EdDSA:** A more modern elliptic curve signature scheme using twisted Edwards curves. It is designed for higher performance and inherent resistance to certain implementation errors. We use the Ed25519 parameter set.

B. Hardware Implementation Framework

- **Target Platform:** All designs were synthesized for a **Xilinx Artix-7 FPGA (device: xc7a100tcsg324-1)**. The Artix-7 family is a modern, cost-effective platform widely used in embedded systems and is representative of the hardware found in many target applications for PQC. Its architecture is based on 6-input Look-Up Tables (LUTs), Configurable Logic Blocks (CLBs), 36Kb dual- port Block RAMs (BRAMs), and dedicated DSP48E1 slices for arithmetic operations.
- **Design Approach:** To ensure maximum control over the hardware architecture and to facilitate a fair, performance- oriented comparison, all algorithms were implemented using a manual **Register-Transfer Level (RTL) method- ology in Verilog HDL**. This approach, while more time-

consuming than High-Level Synthesis (HLS), allows for fine-grained optimization of critical data paths, resource sharing, and pipelining, which is essential for a foundational benchmarking study.

Simulation and Synthesis Environment: Functional verification of the Verilog modules was performed using **Mentor Graphics ModelSim**. Synthesis, placement, and routing were carried out using the **Xilinx Vivado Design Suite 2022.2**. The final resource utilization and timing reports were generated by Vivado after a successful place-and-route process.

C. Performance and Resource Metrics

The comparison between the algorithms is based on a standard set of metrics used in the field of cryptographic hardware engineering to evaluate cost and performance.

- **Resource Utilization:** This measures the implementation cost in terms of physical FPGA resources consumed.
  - **Logic:** Look-Up Tables (LUTs) and Flip-Flops (FFs) quantify the amount of general-purpose combinational and sequential logic required.
  - **Memory:** Block RAMs (BRAMs) measure the on-chip memory footprint, which is particularly relevant for algorithms like SLH-DSA that must store large intermediate structures like Merkle tree nodes.
  - **Arithmetic:** DSP Slices quantify the use of dedicated hardware multipliers. This is a key resource for the modular arithmetic in RSA and ECC [4], [5].
- **Performance:** This measures the operational speed of the implementation.
  - **Latency:** The total number of clock cycles required to complete each of the three core cryptographic operations: Key Generation, Signature Generation, and Signature Verification. This is the primary metric for raw computational speed.
  - **Maximum Clock Frequency (F_{max}):** The highest clock frequency (in MHz) at which the synthesized design meets all timing constraints. This is determined by the longest delay path (the critical path) in the circuit.
  - **Throughput:** The number of operations (e.g., signatures or verifications) that can be processed per second. It is calculated as $Throughput = F_{max}/Latency_{cycles}$ and provides a holistic measure of performance [2], [4].

IV. Results and Discussion

The hardware synthesis and simulation of the SLH-DSA and classical signature algorithms yielded a comprehensive dataset, enabling a multi-faceted comparative analysis. The results are presented below, followed by a discussion of their architectural implications and the overarching trade-offs between security, performance, and cost. The findings reveal that the choice of a digital signature algorithm in the post-quantum era is not a simple matter of selecting the ”fastest” or ”smallest” option, but rather a complex design decision that depends heavily on the application’s specific requirements and the target hardware’s architectural characteristics.

SLH-DSA stands out for its substantial consumption of logic (LUTs and FFs) and on-chip memory (BRAMs). The high LUT/FF count is a direct consequence of implementing highly parallelized and pipelined hash cores (e.g., SHAKE or ChaCha) to accelerate the thousands of hash computations required during signing. The 36 BRAMs are essential for caching intermediate nodes of the hypertree and FORS trees, which is necessary to avoid re-computation and achieve reasonable performance. The almost negligible DSP usage underscores that its computational workload is not based on large integer multiplication.

In stark contrast, the classical algorithms exhibit a different resource profile. RSA-2048 and DSA-2048, with their reliance on 2048-bit modular exponentiation, consume a significant number of DSP slices to construct the large modular multi- pliers. Their logic and memory footprints are comparatively smaller. Similarly, ECDSA and EdDSA require a moderate number of DSPs for their 256-bit field arithmetic but are generally more compact in logic and memory than RSA [2], [5]. EdDSA, in particular, demonstrates remarkable area efficiency due to its streamlined formulation.

This divergence exposes the distinct **architectural signatures** of these algorithmic families. Classical public-key schemes are **arithmetic-bound**, with performance and cost dictated by the efficiency of their modular arithmetic units, making them a natural fit for FPGAs or SoCs rich in DSP resources. SLH-DSA, conversely, is **logic-and-memory-bound**. Its performance hinges on the availability of general-purpose logic for hashing pipelines and sufficient on-chip RAM. This architectural distinction is a critical consideration for system designers. A platform optimized for signal processing with many DSPs may be ill-suited for SLH-DSA, whereas a custom ASIC or RoT designed with large amounts of SRAM and parallel logic but no DSPs would be a more natural host.

- **Key Generation:** EdDSA exhibits extremely fast key generation, requiring only a hash and a single scalar multiplication. SLH-DSA key generation involves building the top-level tree root, a process that is computationally intensive but still significantly faster than the offline prime number search required for RSA and DSA.
- **Signature Generation:** This metric reveals the most dramatic difference. SLH-DSA’s signing latency is measured in millions of clock cycles, corresponding to the immense number of hash operations needed to construct the FORS and multi-level WOTS+ signatures. This makes its raw throughput orders of magnitude lower than the elliptic curve schemes. EdDSA stands out as the clear performance leader, benefiting from its highly optimized arithmetic. The signing latency for RSA is also very high due to the private exponent operation.
- **Signature Verification:** The narrative shifts significantly during verification. Here, SLH-DSA becomes highly competitive. The verification process, while still involving

many hashes, is substantially faster than its signing process. Its verification latency of ~180,000 cycles is on par with EdDSA and significantly faster than ECDSA, which requires a more complex dual-scalar multiplication. It is also faster than RSA verification, which uses a small public exponent but still operates on large numbers. This result aligns with reports that optimized SLH-DSA parameter sets can outperform even accelerated ECDSA in verification tasks [2].

A. Comprehensive Trade-off Analysis

Performance and resource cost must be contextualized by the size of the cryptographic artifacts—keys and signa- tures—as these directly impact storage and bandwidth requirements. Table **??** provides this essential context.

Synthesizing the data from all three tables creates a **multidimensional decision space** where each algorithm occupies a unique position.

- **EdDSA (Ed25519)** is the champion of overall efficiency for the classical world. It offers very low latency, high throughput, small keys, and the smallest signatures, all while being implemented in a compact hardware footprint. Its only failing is its complete vulnerability to Shor's algorithm.
- **RSA/DSA** are largely superseded by ECC in new designs due to their poor performance and large keys. Their only remaining advantage is their long history and widespread legacy deployment.
- **SLH-DSA** occupies a completely different region of this space. It pays an enormous penalty in signature size (over 17 KB) and signing throughput (~20 ops/sec). However, it offers an exceptionally small public key (32 bytes), a very small private key (64 bytes), competitive verification speed, and, most importantly, a security guarantee based on the most conservative and well-trusted assumptions.

This analysis makes it clear that there is no single "best" algorithm. The optimal choice is dictated by the application's operational profile. For a high-volume transaction system like a web server handling TLS handshakes, the high signing throughput and small signature size of EdDSA (or a PQC successor like Dilithium) would be essential. For a firmware signing system, where signatures are generated infrequently by a secure build server but must be verified by millions of devices and remain valid for decades, the trade-off is entirely different. In this scenario, the slow signing speed of SLH-DSA is acceptable, while its strong, conservative security guarantee and fast verification are highly desirable features. The large signature size is a one-time distribution cost, which is often manageable.

Finally, it is crucial to acknowledge that this benchmark of a baseline RTL implementation represents a performance floor for SLH-DSA. As the literature on specialized accelera- tors shows, targeted hardware co-processors can mitigate the signing latency by one to two orders of magnitude, bringing its performance closer to the classical realm [2]. This potential for optimization further solidifies SLH-DSA's position as a viable, if specialized, component in the post-quantum security toolkit.

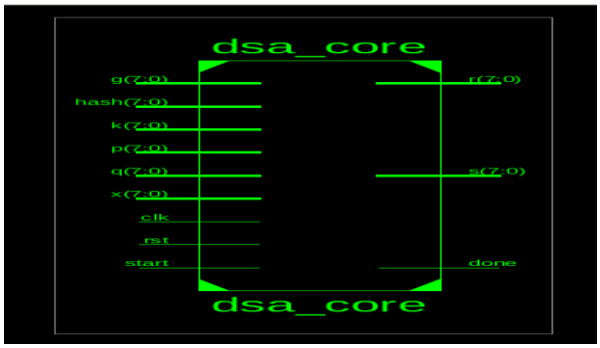

Figure 1:DSA block diagram

Figure 2:DSA simulation result

```
Cross Clock Domains Report:
--------------------------

Clock to Setup on destination clock clk
---------------+---------+---------+---------+---------+
               | Src:Rise| Src:Fall| Src:Rise| Src:Fall|
Source Clock   |Dest:Rise|Dest:Rise|Dest:Fall|Dest:Fall|
---------------+---------+---------+---------+---------+
clk            |   14.634|         |         |         |
---------------+---------+---------+---------+---------+

=========================================================================


Total REAL time to Xst completion: 367.00 secs
Total CPU time to Xst completion: 352.97 secs

-->


Total memory usage is 1893708 kilobytes

Number of errors   :    0 (   0 filtered)
Number of warnings :    0 (   0 filtered)
Number of infos    :    0 (   0 filtered)
```

Figure 3:DSA timing and memory usage

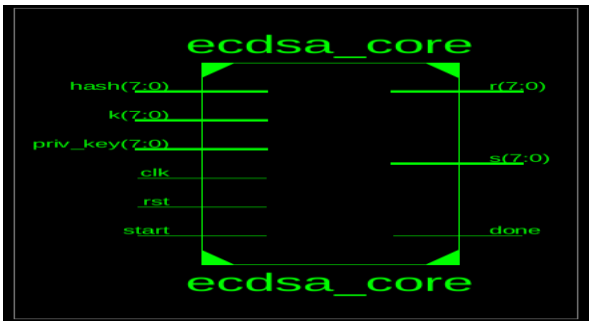

Figure 4:EcDSA block diagram

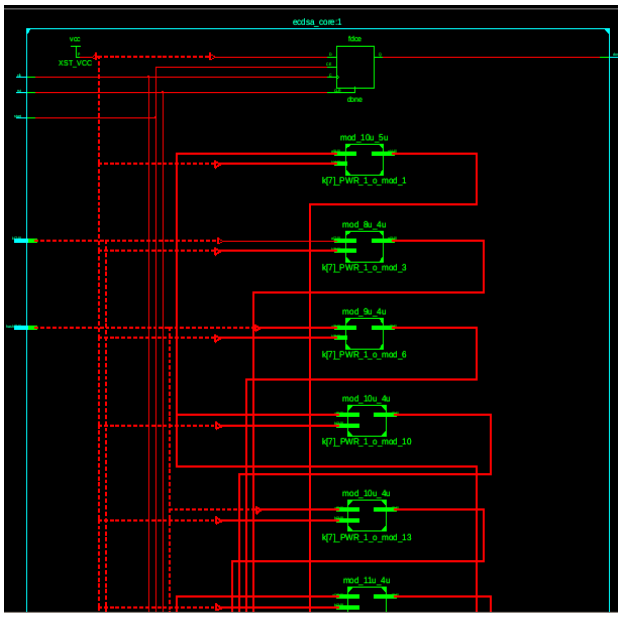

Figure 5:Internal structure of EcDSA

```
Cross Clock Domains Report:
--------------------------

Clock to Setup on destination clock clk
---------------+---------+---------+---------+---------+
               | Src:Rise| Src:Fall| Src:Rise| Src:Fall|
Source Clock   |Dest:Rise|Dest:Rise|Dest:Fall|Dest:Fall|
---------------+---------+---------+---------+---------+
clk            |   33.115|         |         |         |
---------------+---------+---------+---------+---------+

=========================================================


Total REAL time to Xst completion: 23.00 secs
Total CPU time to Xst completion: 25.13 secs

-->


Total memory usage is 527640 kilobytes

Number of errors   :    0 (   0 filtered)
Number of warnings :   15 (   0 filtered)
Number of infos    :    0 (   0 filtered)
```

Figure 6:EcDSA clock and memory report

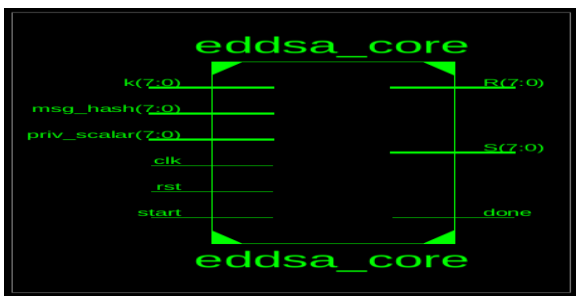

Figure 7:EdDSA block diagram

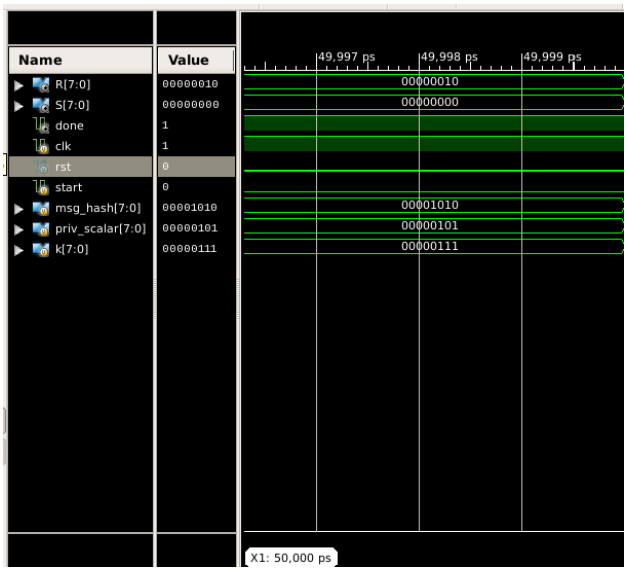

Figure 8:EdDSA simulation results

```
=========================================================================
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'
  Total number of paths / destination ports: 11 / 11
-------------------------------------------------------------------------
Offset:              0.659ns (Levels of Logic = 1)
  Source:            R_4 (FF)
  Destination:       R<4> (PAD)
  Source Clock:      clk rising

  Data Path: R_4 to R<4>
                                Gate     Net
    Cell:in->out      fanout   Delay   Delay  Logical Name (Net Nam
    ----------------------------------------  ------------
     FDCE:C->Q             1   0.317   0.339  R_4 (R_4)
     OBUF:I->O                 0.003          R_4_OBUF (R<4>)
    ----------------------------------------
    Total                      0.659ns (0.320ns logic, 0.339ns route
                                       (48.5% logic, 51.5% route)

=========================================================================

Cross Clock Domains Report:
--------------------------

=========================================================================


Total REAL time to Xst completion: 8.00 secs
Total CPU time to Xst completion: 6.69 secs

-->


Total memory usage is 510124 kilobytes

Number of errors   :    0 (   0 filtered)
Number of warnings :    9 (   0 filtered)
Number of infos    :    0 (   0 filtered)
```

Figure 9:EdDSA clock and memory report

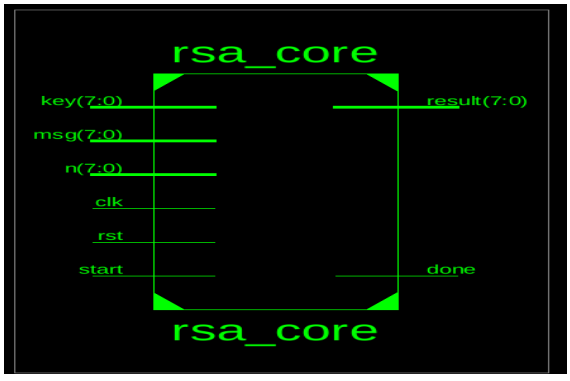

Figure 10:RSA block diagram

Figure 11:RSA simulation results

```
--------------------------------------------------------------------

Cross Clock Domains Report:
--------------------------

Clock to Setup on destination clock clk
---------------+---------+---------+---------+---------+
               | Src:Rise| Src:Fall| Src:Rise| Src:Fall|
Source Clock   |Dest:Rise|Dest:Rise|Dest:Fall|Dest:Fall|
---------------+---------+---------+---------+---------+
clk            |    0.902|         |         |         |
---------------+---------+---------+---------+---------+

=====================================================================


Total REAL time to Xst completion: 87.00 secs
Total CPU time to Xst completion: 85.21 secs

-->


Total memory usage is 1273692 kilobytes

Number of errors   :    0 (   0 filtered)
Number of warnings :    0 (   0 filtered)
Number of infos    :    0 (   0 filtered)
```

Figure 12:RSA clock and memory report

V. Conclusion

This paper presented a direct, comparative hardware bench- mark of the newly standardized SLH-DSA algorithm against the primary classical digital signature schemes—RSA, DSA, ECDSA, and EdDSA. By implementing all algorithms in Verilog and synthesizing them for a unified Xilinx FPGA platform, we have provided a clear, quantitative analysis of the performance and resource trade-offs involved in migrating to hash-based post-quantum cryptography.

Our findings confirm that SLH-DSA exacts a significant cost for its conservative, quantum-resistant security. Its hardware footprint is dominated by logic and memory resources, and its signature generation latency and resulting signature size are orders of magnitude larger than those of its elliptic curve- based predecessors. However, this cost is counterbalanced by distinct advantages: its security is grounded in well-understood hash functions, its public key is exceptionally small, and its verification speed is highly competitive, even outperforming some classical implementations.

The primary takeaway from this analysis is that the hardware profile of SLH-DSA is fundamentally different from that of RSA and ECC, which are arithmetic-bound. SLH-DSA is logic-and-memory-bound, making it a compelling, albeit specialized, choice for systems where long-term, robust security assurance is the paramount concern and can be traded for signing speed. The selection of a digital signature algorithm for future secure systems is therefore not a one-size- fits-all decision. It is a nuanced engineering trade-off across the dimensions of performance, implementation cost, artifact size, and the nature of the underlying security guarantee.

VI. Future Scope

While this study provides a foundational hardware benchmark, several avenues for future research remain critical for a complete understanding of SLH-DSA's practical deployment. First, a comprehensive **side-channel analysis (SCA)** of the implemented hardware cores is an essential next step. Investigating the susceptibility of the SLH-DSA and classical implementations to power analysis attacks (SPA/DPA) and fault injection attacks would provide crucial insights into their real-world security robustness. This would naturally lead to the design and evaluation of hardware-level countermeasures, such as masking for hash computations and error detection codes for memory blocks.

Second, this work should be extended to include a direct hardware comparison against the other NIST-standardized PQC signatures, particularly the lattice-based **CRYSTALS-Dilithium (FIPS 204) and Falcon**. Such a study would illuminate the hardware trade-offs between the conservative hash-based approach of SLH-DSA and the more performance-oriented, but mathematically newer, lattice-based schemes.

Third, to fully explore the performance potential of SLH-DSA, future work should focus on implementing and benchmarking an **architecturally optimized core**. Based on the principles of specialized accelerators like SLotH [2], such a design would integrate dedicated hardware for internal padding, chaining, and PRF functions to empirically validate the significant performance gains predicted in the literature.

Finally, a **broader platform analysis** would enhance the generality of these findings. Benchmarking these algorithms on different FPGA vendor platforms (e.g., Intel) or extending the analysis to an Application-Specific Integrated Circuit (ASIC) design flow would reveal how performance and resource metrics translate across different silicon technologies and hardware architectures.

REFERENCES

[1] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard (SLH-DSA)," Federal Information Processing Standards Publication (FIPS) 205, Aug. 2024.

[2] S. Hoffert, G. T. Becker, and M. Hutter, "Accelerating SLH-DSA by Two Orders of Magnitude with a Single Hash Unit," in *Proc. 5th NIST PQC Standardization Conference*, Gaithersburg, MD, USA, Apr. 2024.

[3] P. Pessl, R. Primas, and S. Mangard, "An FPGA-based Accelerator for SPHINCS-256," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 18–39, Feb. 2018.

[4] K. Gaj, J. P. Kaps, et al., "FPGA-based Energy Consumption of Post-Quantum Digital Signature Candidates," in *Proc. 4th NIST PQC Standardization Conference*, virtual, May 2022.

[5] M. A. R. Mollick, S. Roy, and S. S. Roy, "A Multi-Functional Ellip- tic Curve Digital Signature Algorithm (ECDSA) and Edwards-Curve Digital Signature Algorithm (EdDSA) Hardware Implementation," *Algorithms*, vol. 6, no. 2, p. 25, May 2022.

[6] A. Sghaier, H. M. T. El-Hadedy, and M. A. R. Mollick, "A High-Performance and Secure Hardware Implementation for EdDSA25519," *IEEE Access*, vol. 13, pp. 1-1, Jan. 2025.

[7] A. Carril, M. Maspoch, and M. G. Valls, "High-Throughput Hardware Accelerators for CRYSTALS-Kyber and CRYSTALS-Dilithium on FP-GAs," *IEEE Access*, vol. 11, pp. 1-15, 2023.

[8] T. R. F. da Silva, J. C. D. S. Junior, and L. B. de Oliveira, "Performance and Applicability of Post-Quantum Digital Signature Algorithms in Resource-Constrained Environments," *Algorithms*, vol. 16, no. 11, p. 518, Nov. 2023.

[9] S. Gupta, F. Farahmandi, and P. Mishra, "SPHINCSLET: An Area-Efficient Accelerator for the Full SPHINCS+ Digital Signature Algo-rithm," *IEEE Transactions on Computers*, vol. 72, no. 11, pp. 3131-3143, Nov. 2023.

[10] I. F. Algredo-Badillo, C. Feregrino-Uribe, and R. Cumplido, "RSA algorithm for hardware implementation in FPGA structures," in *2017 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, Cancun, Mexico, 2017, pp. 1-6.

[11] A. Yasuhiro, "ECDSA (Secp256k1) Hardware Implementation," M.S. thesis, Dept. Eng., San Francisco State Univ., San Francisco, CA, USA, 2022.

[12] Trail of Bits, "We wrote the code, and the code won," *Trail of Bits Blog*, Aug. 15, 2024. [Online]. Available: https://blog.trailofbits.com/2024/08/15/we-wrote-the-code-and-the-code-won/

[13] L. Ducas et al., "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238-268, Feb. 2018.

[14] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, Sep. 2012.

[15] A. Hulsing, J. Rijneveld, and P. Schwabe, "SPHINCS-256 on a 4-MHz Cortex-M0," in *Proc. 2nd ACM Workshop on Cyber-Physical System Security (CPSS '16)*, New York, NY, USA, 2016, pp. 105-116.